P.Vishwa Nandan(192473015)

Q2 – When Is ArrayList an Appropriate Engineering Choice?

Marks: 4

This question is about comparing three Java collection concepts:

1. ArrayList
2. LinkedList
3. ArrayDeque

Problem Context – Music Playlist Editor

Imagine a music-streaming application where users can create playlists.

Users should be able to:

- Add songs at the end.
- Insert songs at specific positions.
- Remove songs from the beginning or middle.
- Directly access the k-th song.
- Iterate through the complete playlist.

The application currently uses ArrayList<Song>.

We need to investigate whether ArrayList is the best choice for all these operations. Pasted text(6)

---

1. Understanding ArrayList and LinkedList

A. ArrayList

ArrayList stores elements using a dynamically resizable array.

Example:

ArrayList<String> songs = new ArrayList<>();

songs.add("Song 1");

songs.add("Song 2");

songs.add("Song 3");

Advantages:

- Fast direct access using index.
- Adding at the end is generally fast.
- Easy to iterate through elements.

Disadvantages:

- Inserting at the beginning requires shifting elements.
- Removing from the beginning requires shifting elements.

B. LinkedList

LinkedList stores elements as separate nodes. Each node contains data and references to the previous and next nodes.

Example:

LinkedList<String> songs = new LinkedList<>();

songs.add("Song 1");

songs.add("Song 2");

songs.add("Song 3");

Advantages:

- Fast insertion and removal at both ends.
- Does not require shifting elements.
- Useful for sequential operations.

Disadvantages:

- Direct access to the k-th element is slow.
- Each node requires additional memory.
- Searching for a middle position requires traversal.

---

2. Prediction Before Implementation

Before conducting experiments, we can predict the performance.

Operation	**ArrayList**	**LinkedList**
Add at end	O(1) amortized	O(1)

Operation	ArrayList	LinkedList
Insert at front	O(n)	O(1)
Insert in middle	O(n)	O(n)
Remove at front	O(n)	O(1)
Remove in middle	O(n)	O(n)
Access k-th element	O(1)	O(n)
Iterate entire list	O(n)	O(n)

Here, n represents the number of songs.

Predicted result:

1. ArrayList should perform better for accessing songs by index.
2. LinkedList should perform better for frequent insertion and removal at the beginning.
3. Both should support efficient sequential iteration.
4. ArrayList should generally use less memory per element because LinkedList stores additional node references.

---

3. Design the Experiments

We use three different playlist sizes.

Experiment 1: 1,000 songs.

Experiment 2: 10,000 songs.

Experiment 3: 50,000 songs.

For each size, we compare the following operations:

1. Adding songs at the end.
2. Inserting songs at the beginning.
3. Inserting songs in the middle.
4. Removing songs from the beginning.
5. Removing songs from the middle.
6. Accessing the k-th song.
7. Iterating through the complete playlist.

This helps us understand how performance changes as playlist size increases.

Explanation of the Program

We use two methods to create the playlists:

- createArrayList() creates an ArrayList.
- createLinkedList() creates a LinkedList.

The remaining methods measure the time required for each operation.

For example:

list.add(song);

Adds a song at the end.

list.add(0, song);

Adds a song at the beginning.

list.add(list.size() / 2, song);

Adds a song in the middle.

list.remove(0);

Removes the first song.

list.get(n / 2);

Accesses the middle song.

We use System.nanoTime() to measure execution time.

Note: The program uses up to 500 insertion/removal operations per experiment and 1,000 indexed accesses. This keeps the benchmark practical while still showing how the operations scale. Timings vary depending on your laptop and Java environment.

---

5. Experimental Results

After executing the program, record your actual timings in the following tables.

Experiment 1 – 1,000 Songs

Operation	**ArrayList (ms)**	**LinkedList (ms)**
Add at end	Your result	Your result
Insert at front	Your result	Your result

Operation	ArrayList (ms)	LinkedList (ms)
Insert in middle	Your result	Your result
Remove at front	Your result	Your result
Remove in middle	Your result	Your result
Access k-th	Your result	Your result
Iterate	Your result	Your result

Experiment 2 – 10,000 Songs

Operation	ArrayList (ms)	LinkedList (ms)
Add at end	Your result	Your result
Insert at front	Your result	Your result
Insert in middle	Your result	Your result
Remove at front	Your result	Your result
Remove in middle	Your result	Your result
Access k-th	Your result	Your result
Iterate	Your result	Your result

Experiment 3 – 50,000 Songs

Operation	ArrayList (ms)	LinkedList (ms)
Add at end	Your result	Your result
Insert at front	Your result	Your result
Insert in middle	Your result	Your result
Remove at front	Your result	Your result
Remove in middle	Your result	Your result
Access k-th	Your result	Your result
Iterate	Your result	Your result

Important: Fill these tables using the actual output from VS Code. Do not invent experimental measurements.

---

6. When Is ArrayList a Good Engineering Choice?

ArrayList is a reasonable choice when:

1. Songs are frequently accessed using their index.
2. Songs are mainly added at the end.
3. Users frequently browse and iterate through playlists.
4. Insertion and deletion at the beginning are relatively uncommon.

ArrayList is preferable for a normal music playlist because users frequently select songs by position and browse the playlist.

When Should We Stop Recommending ArrayList?

We should reconsider ArrayList when:

1. Songs are frequently inserted at the beginning.
2. Songs are frequently removed from the beginning.
3. The application performs many front-based queue operations.
4. Random access is not an important requirement.

For these situations, LinkedList or ArrayDeque may be more suitable depending on the required operations.

---

7. Changed Condition – Up Next Queue

Now the application introduces a new feature called **Up Next**.

In this feature:

- Songs are frequently added at the beginning.
- Songs are frequently added at the end.
- Songs are frequently removed from the beginning.

For this requirement, ArrayDeque is a good choice.

Why Is ArrayDeque Suitable?

1. Adding songs at the beginning is efficient.

2. Adding songs at the end is efficient.
3. Removing songs from the beginning is efficient.
4. It uses a resizable circular array.
5. It avoids the per-node memory overhead of LinkedList.

Therefore, ArrayDeque is suitable for the Up Next queue.

---

8. Alternative Concept – Complexity and Space Comparison

The assignment asks us to compare ArrayList, LinkedList and ArrayDeque for different operations.

A. Time Complexity Comparison

Operation	**ArrayList**	**LinkedList**	**ArrayDeque**
Add at end	O(1) amortized	O(1)	O(1) amortized
Add at front	O(n)	O(1)	O(1) amortized
Insert in middle	O(n)	O(n)	Not directly supported
Remove at front	O(n)	O(1)	O(1)
Remove in middle	O(n)	O(n)	Not directly supported
Get k-th element	O(1)	O(n)	O(n)
Iterate entire list	O(n)	O(n)	O(n)

Note: LinkedList insertion and removal at a middle position are O(1) once the required node or ListIterator position is already located. Finding that position by index takes O(n).

ArrayDeque does not provide direct indexed access or efficient middle insertion and removal.

B. Space Complexity

ArrayList:

Stores element references in a dynamically resizable array.

Space complexity: O(n).

It may maintain some unused capacity.

LinkedList:

Each node stores:

- Element reference.
- Previous node reference.
- Next node reference.

Space complexity: O(n).

It requires more memory per element because of node overhead.

ArrayDeque:

Stores element references in a circular array.

Space complexity: O(n), with some additional unused capacity.

It avoids the extra previous and next references required by LinkedList.

C. Plain Array Approach

A playlist can also be stored in a normal Java array.

Example:

String[] songs = new String[5];

songs[0] = "Song A";

songs[1] = "Song B";

songs[2] = "Song C";

To insert at the beginning, we must shift existing elements one position to the right.

Example:

Before insertion:

[A] [B] [C] [D]

After inserting X:

[X] [A] [B] [C] [D]

The time complexity for front insertion is O(n).

Similarly, removing an element requires shifting elements to fill the empty position.

Operation	Plain Array
Add at end (space available)	O(1)
Insert at front	O(n)
Insert in middle	O(n)
Remove at front	O(n)
Remove in middle	O(n)
Get k-th element	O(1)
Iterate	O(n)

A fixed-size array cannot grow automatically. A new larger array must be created when the capacity is exceeded.

Therefore, ArrayList is more convenient for a dynamically changing playlist.

---

9. Final Comparison and Recommendation

For a normal playlist:

ArrayList is recommended because direct access, browsing and adding songs at the end are important operations.

For frequent insertion and deletion at arbitrary positions:

LinkedList may be considered when the application already has a reference to the required node or iterator position. Finding a middle position still requires O(n) traversal.

For the Up Next queue:

ArrayDeque is recommended because it provides efficient insertion and removal at both ends.

Conclusion

ArrayList is a suitable engineering choice when random access and iteration are frequent. LinkedList is useful for operations at both ends or when insertion and deletion occur at known node positions. ArrayDeque is more appropriate for the Up Next queue because it efficiently supports double-ended operations. The final choice depends on the application's usage pattern rather than selecting one collection for every situation.